Name: Keerthi

Program: Azure Data Engineering

PROJECT 5: FABRIC

1. INTRODUCTION:

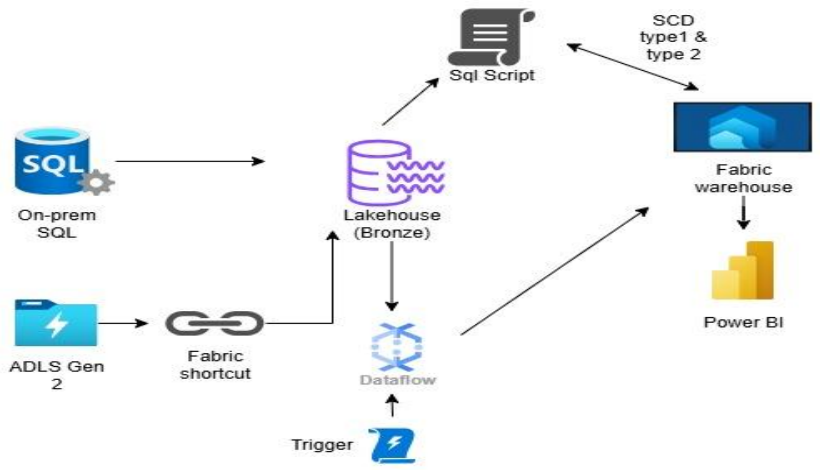
This project demonstrates the implementation of an end-to-end data engineering pipeline using Microsoft Fabric. The objective is to ingest data, transform it using modern data engineering tools, and implement Slowly Changing Dimensions (SCD) Type 1 and Type 2 in a data warehouse environment.

The project simulates real-world data warehouse scenarios where data changes over time and needs to be tracked efficiently. The solution uses Lakehouse architecture for data storage and a Warehouse for analytical processing.

2. ARCHITECTURE:

The architecture follows a Medallion Architecture approach:

- **Bronze Layer:** Raw data ingested into Lakehouse
- **Silver Layer:** Cleaned and transformed data using Dataflow Gen2
- **Gold Layer (Warehouse):** Final analytical tables with SCD implementations



ARCHITECTURE DIAGRAM

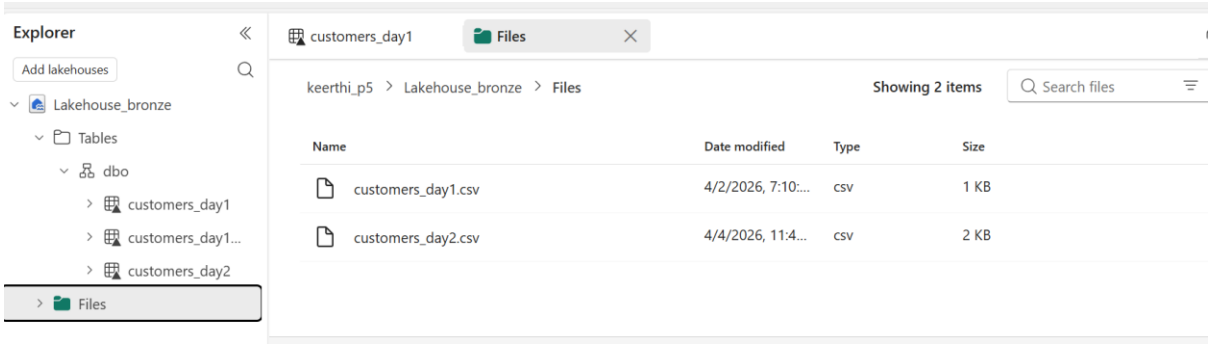
Data Flow:

Source Files → Lakehouse (Bronze) → Dataflow Gen2 (Silver) → Warehouse (Staging) → Dimension Tables (SCD Type 1 & Type 2)

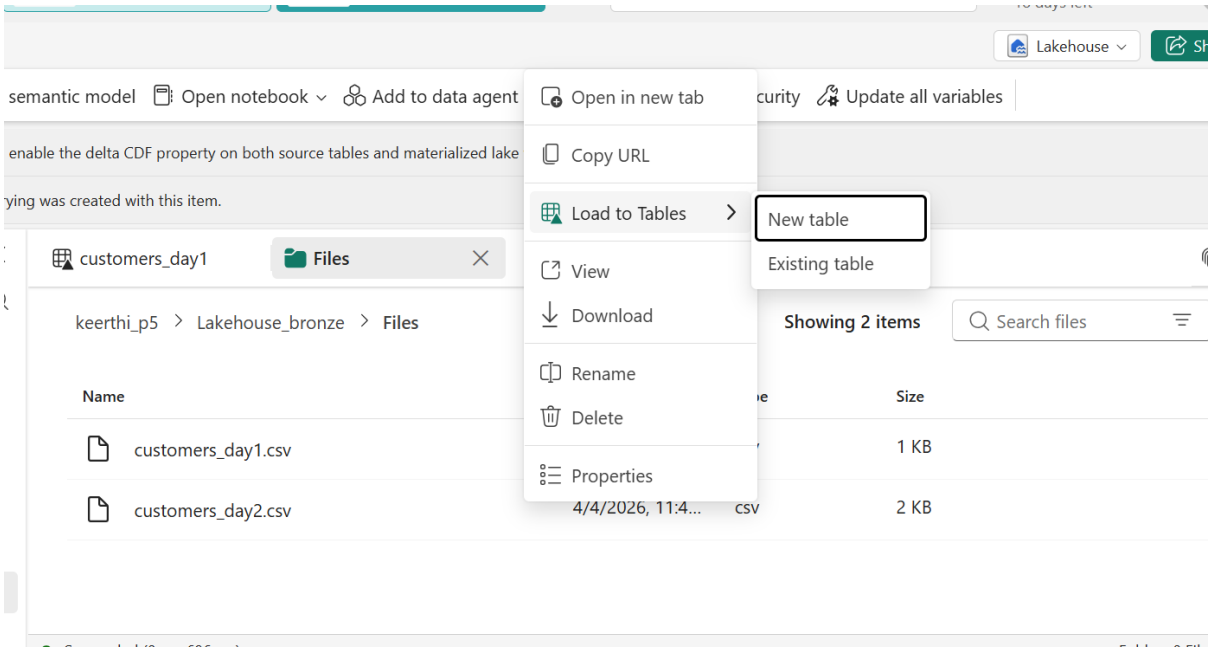
3. DATA INGESTION

Due to the absence of ADLS Gen2 and on-premises SQL connectivity, data was directly uploaded into the Lakehouse using local files.

- Initial dataset (Customer Day 1) uploaded into Lakehouse
- Updated dataset (Customer Day 2) used to simulate changes



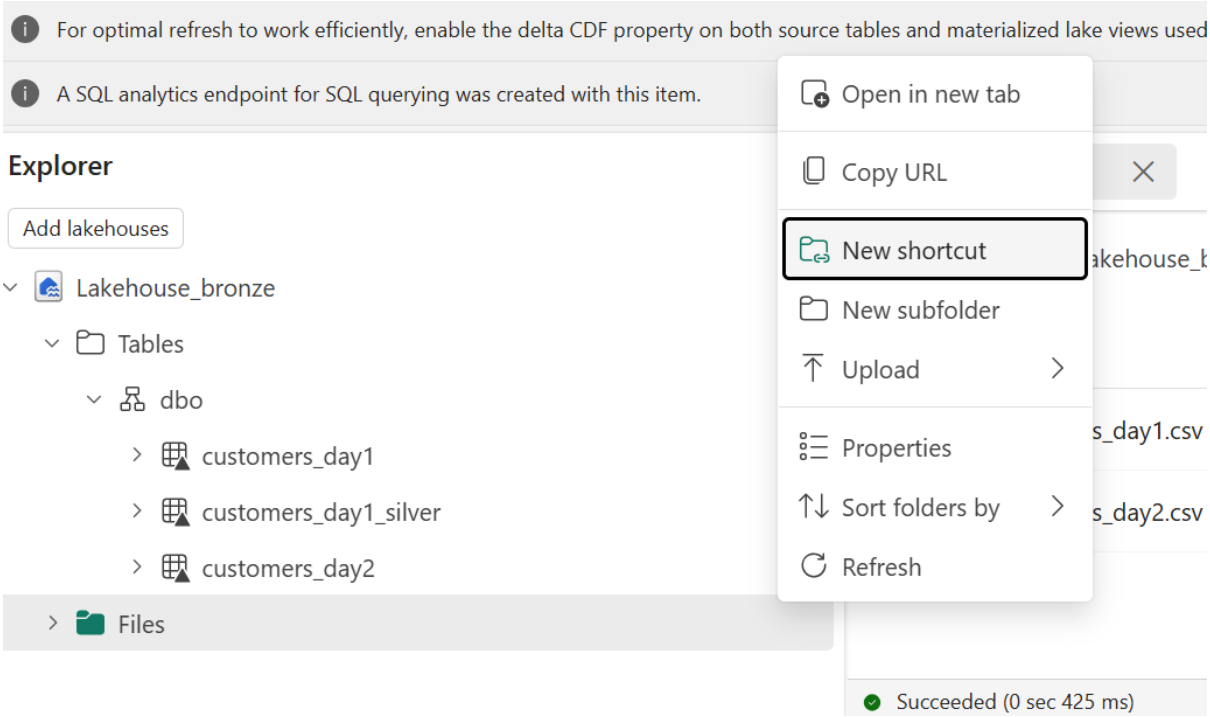
Converting the files uploaded into Tables in Lakehouse:

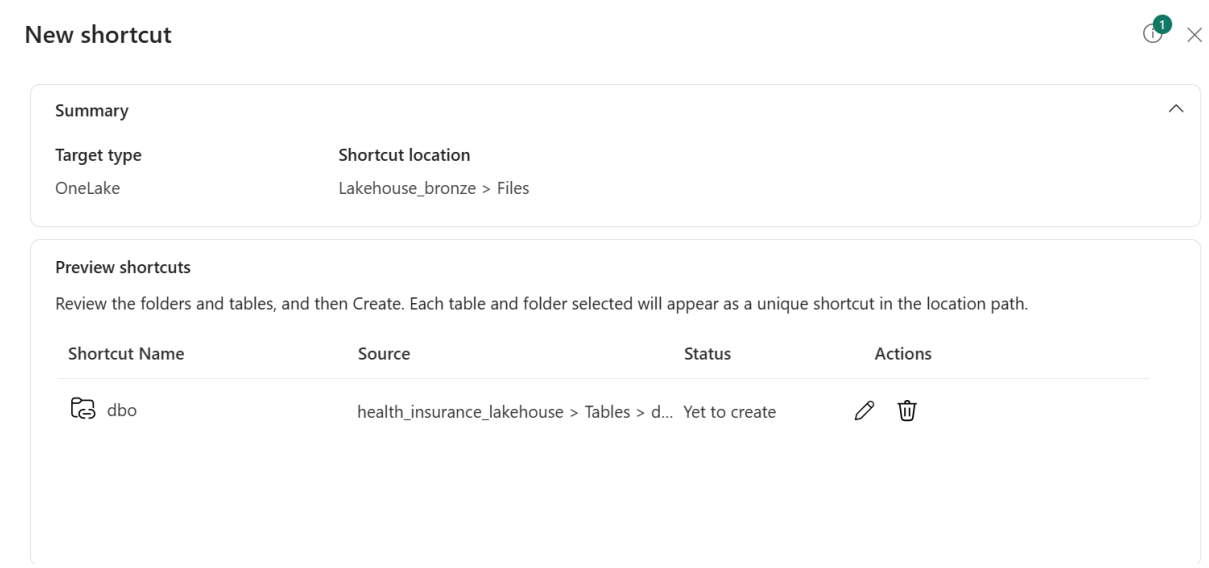


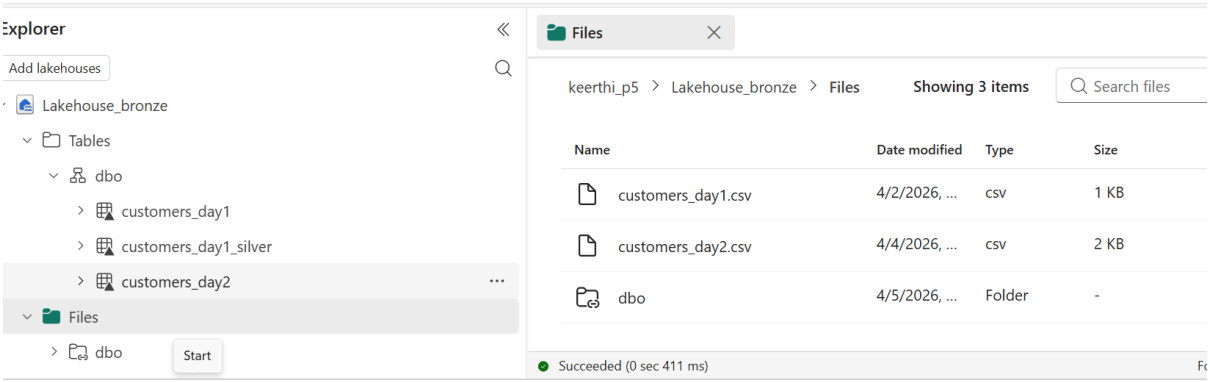
Using Shortcut in Fabric:

Shortcuts are used in the Lakehouse to simulate external data access. A shortcut was created by referencing existing Lakehouse tables using the “New Shortcut” option. This allowed the data to be accessed without duplication and used seamlessly in downstream processes such as data pipelines

and transformations. The shortcut behaves like a regular table while maintaining a link to the original data source, ensuring efficient data access and reduced storage usage.





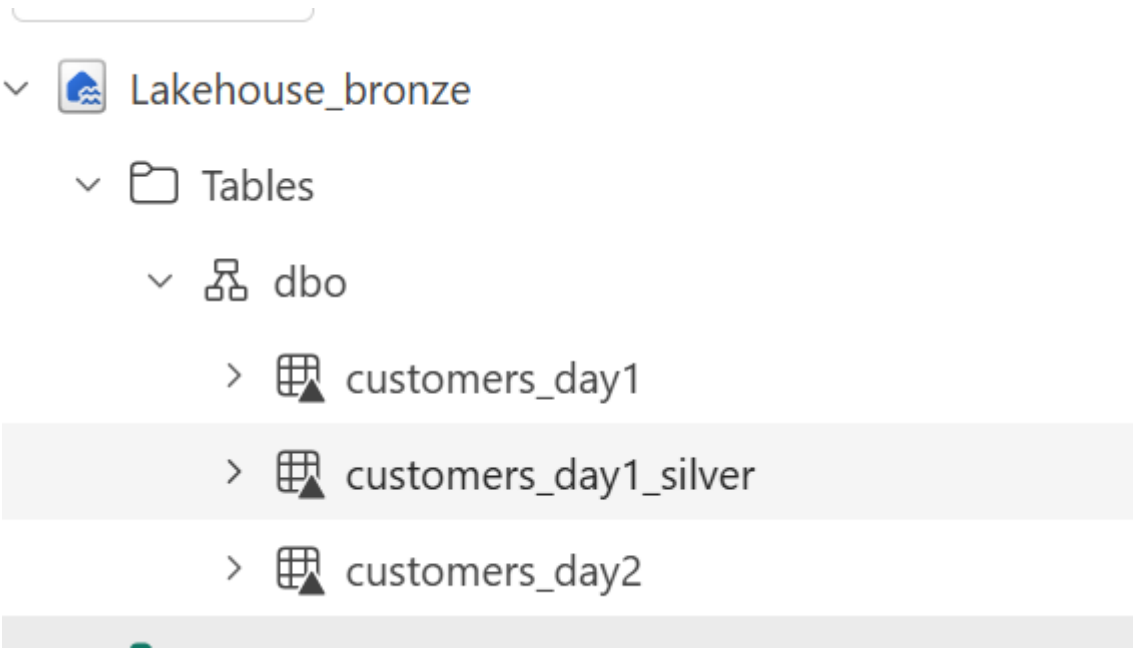


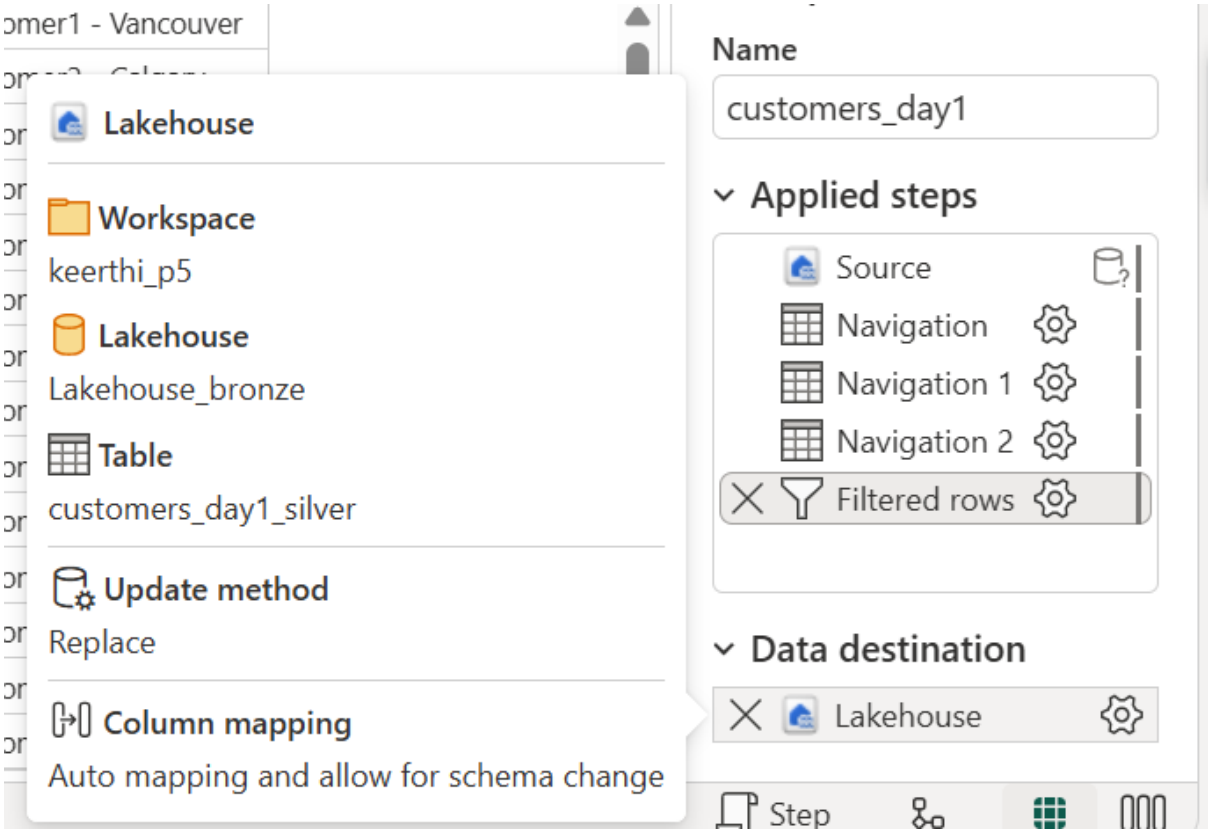
4. DATA TRANSFORMATION (SILVER LAYER):

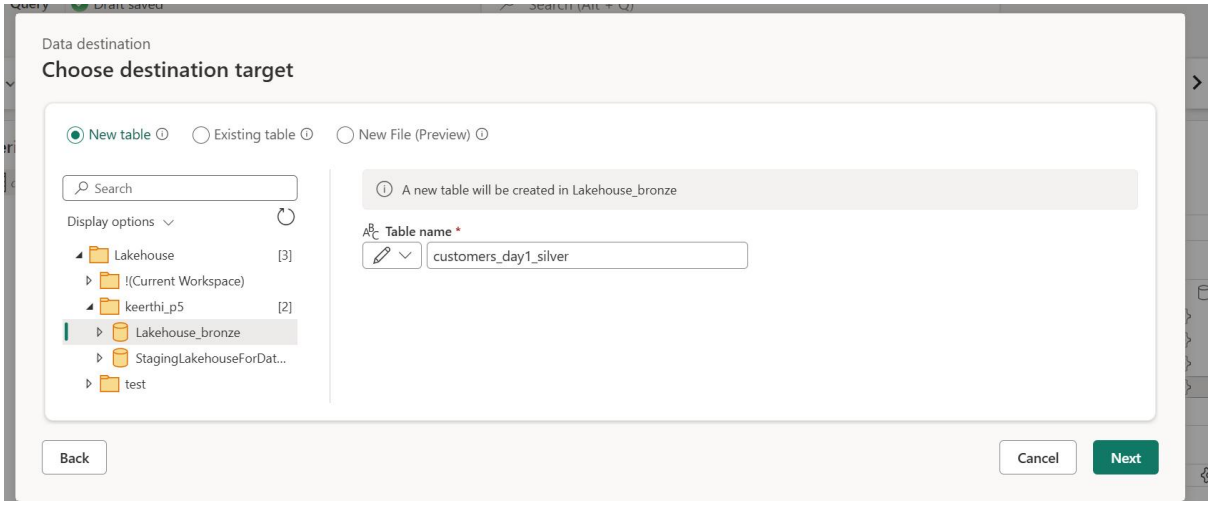
Dataflow Gen2 was used to clean and transform the raw data:

- Removed null values
- Standardized column names
- Ensured correct data types
- Removed duplicates

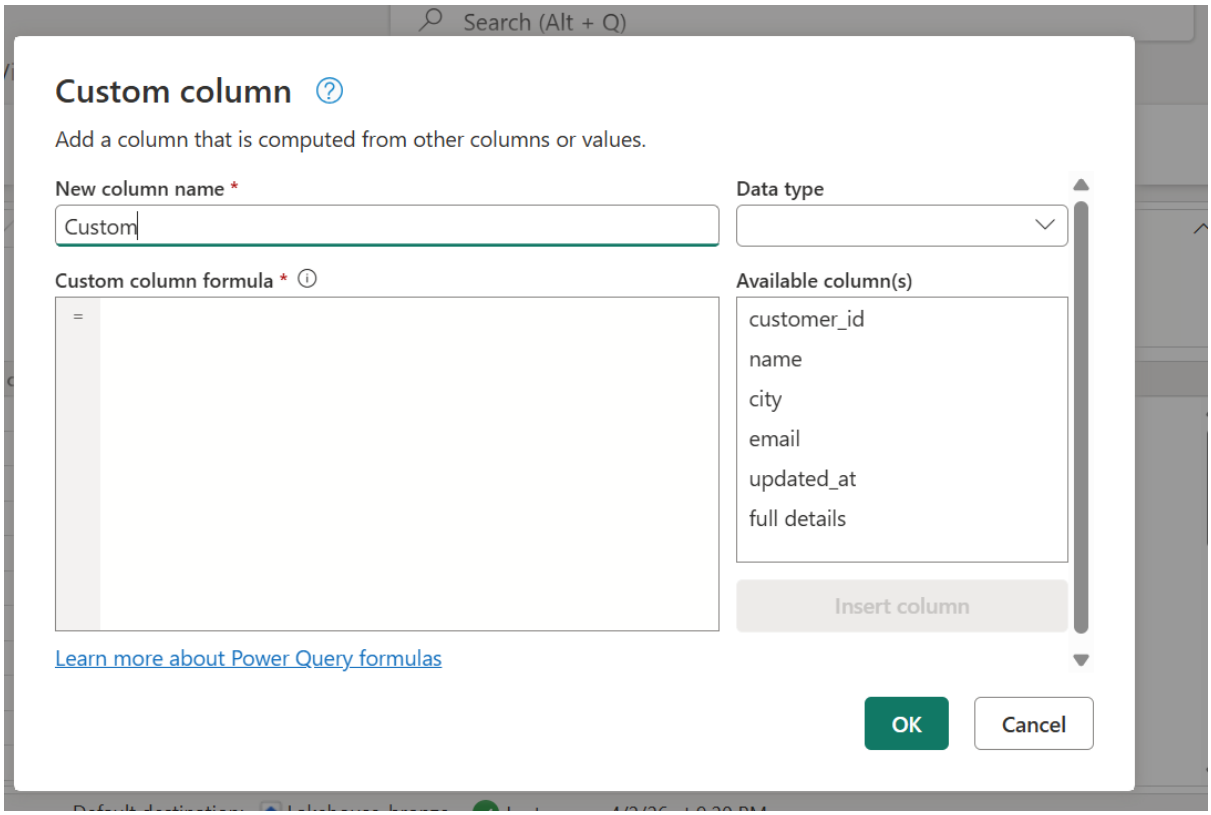
The cleaned data was stored in Lakehouse as Silver tables.



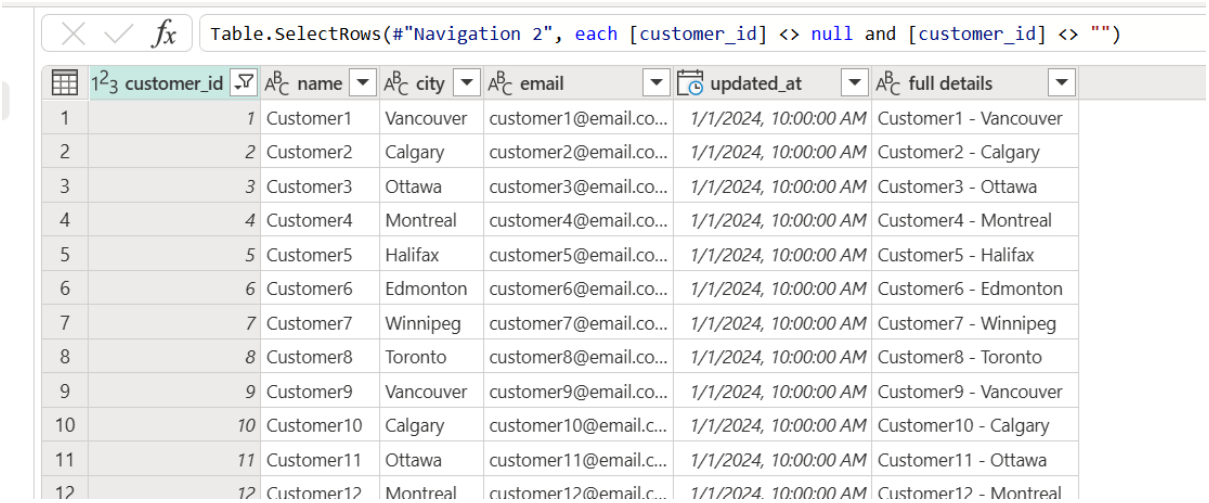

omer1 - Vancouver
Lakehouse
Workspace
keerthi_p5
Lakehouse
Lakehouse_bronze
Table
customers_day1_silver
Update method
Replace
Column mapping
Auto mapping and allow for schema change
Name
customers_day1
Applied steps
Source
Navigation
Navigation 1
Navigation 2
Filtered rows
Data destination
Lakehouse
Step


Data destination
Choose destination target
New table
Existing table
New File (Preview)
Search
Display options
Lakehouse [3]
!(Current Workspace)
keerthi_p5 [2]
Lakehouse_bronze
StagingLakehouseForDat...
test
A new table will be created in Lakehouse_bronze
Table name *
customers_day1_silver
Back
Cancel
Next

To add additional custom column:



Results after applying the transformations:



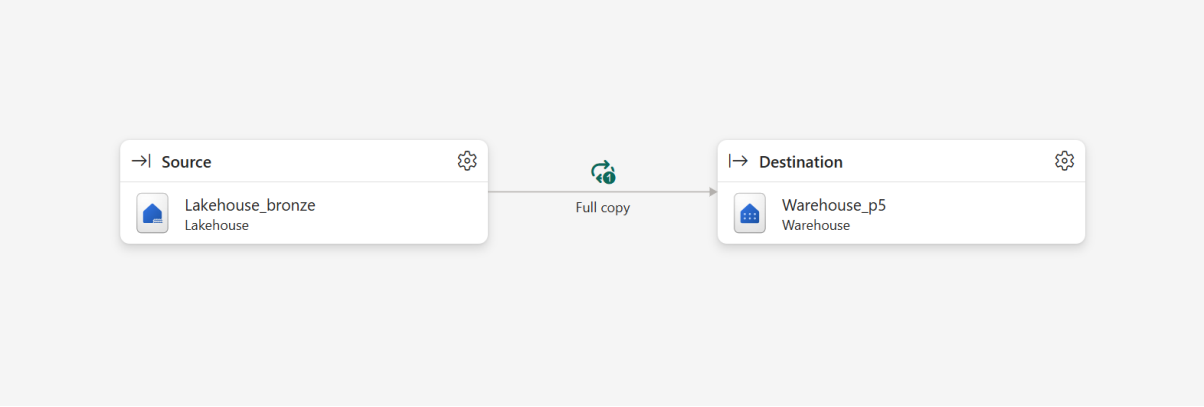
```
Table.SelectRows(#"Navigation 2", each [customer_id] <> null and [customer_id] <> "")
```

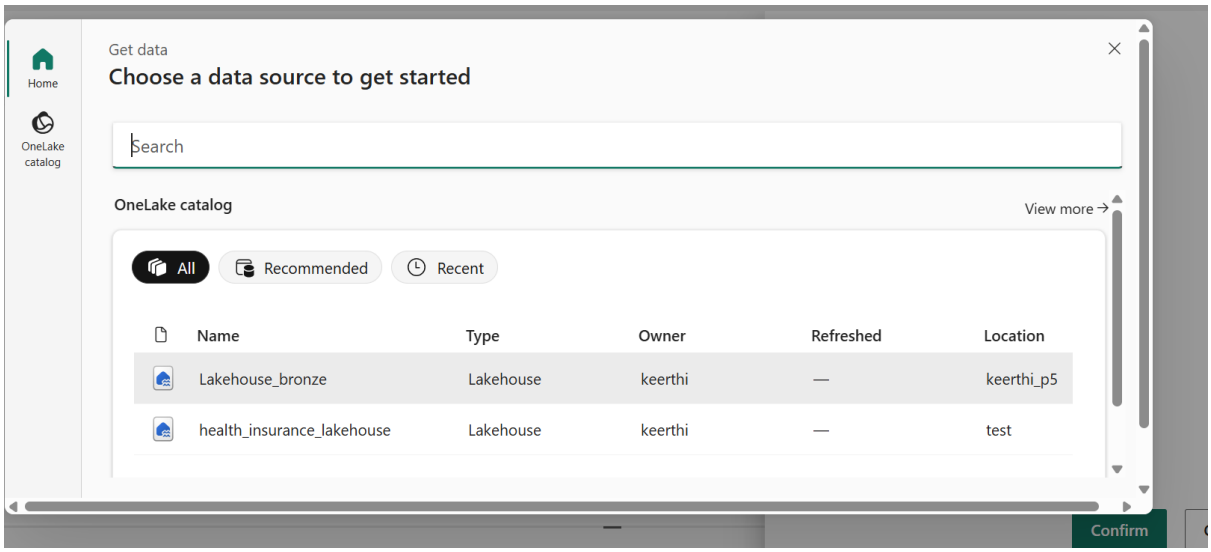
	customer_id	name	city	email	updated_at	full details
1	1	Customer1	Vancouver	customer1@email.co...	1/1/2024, 10:00:00 AM	Customer1 - Vancouver
2	2	Customer2	Calgary	customer2@email.co...	1/1/2024, 10:00:00 AM	Customer2 - Calgary
3	3	Customer3	Ottawa	customer3@email.co...	1/1/2024, 10:00:00 AM	Customer3 - Ottawa
4	4	Customer4	Montreal	customer4@email.co...	1/1/2024, 10:00:00 AM	Customer4 - Montreal
5	5	Customer5	Halifax	customer5@email.co...	1/1/2024, 10:00:00 AM	Customer5 - Halifax
6	6	Customer6	Edmonton	customer6@email.co...	1/1/2024, 10:00:00 AM	Customer6 - Edmonton
7	7	Customer7	Winnipeg	customer7@email.co...	1/1/2024, 10:00:00 AM	Customer7 - Winnipeg
8	8	Customer8	Toronto	customer8@email.co...	1/1/2024, 10:00:00 AM	Customer8 - Toronto
9	9	Customer9	Vancouver	customer9@email.co...	1/1/2024, 10:00:00 AM	Customer9 - Vancouver
10	10	Customer10	Calgary	customer10@email.c...	1/1/2024, 10:00:00 AM	Customer10 - Calgary
11	11	Customer11	Ottawa	customer11@email.c...	1/1/2024, 10:00:00 AM	Customer11 - Ottawa
12	12	Customer12	Montreal	customer12@email.c...	1/1/2024, 10:00:00 AM	Customer12 - Montreal

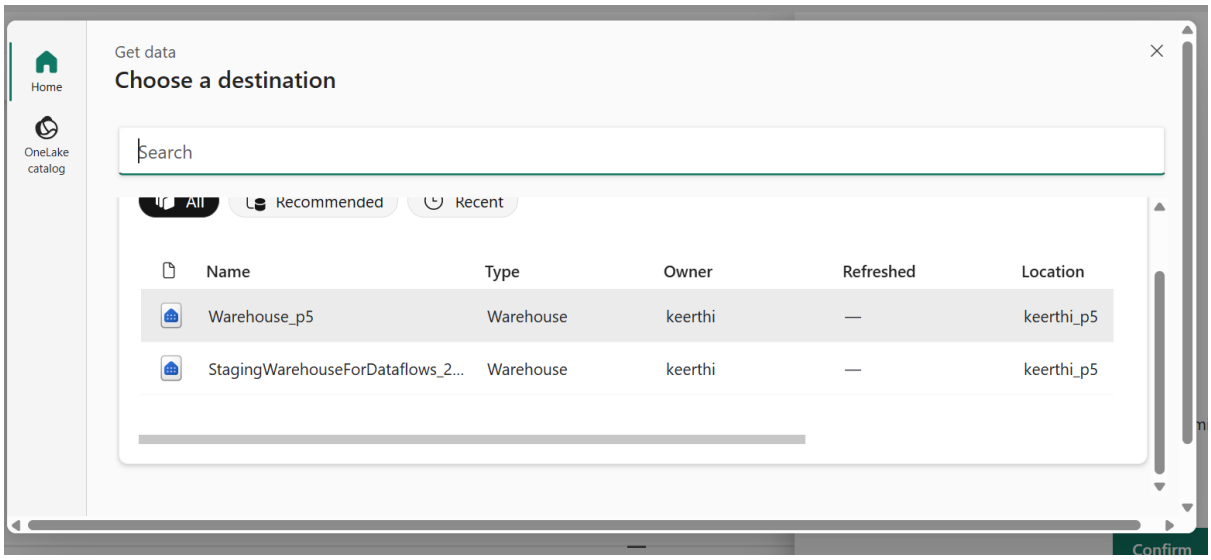
5. DATA LOADING INTO WAREHOUSE:

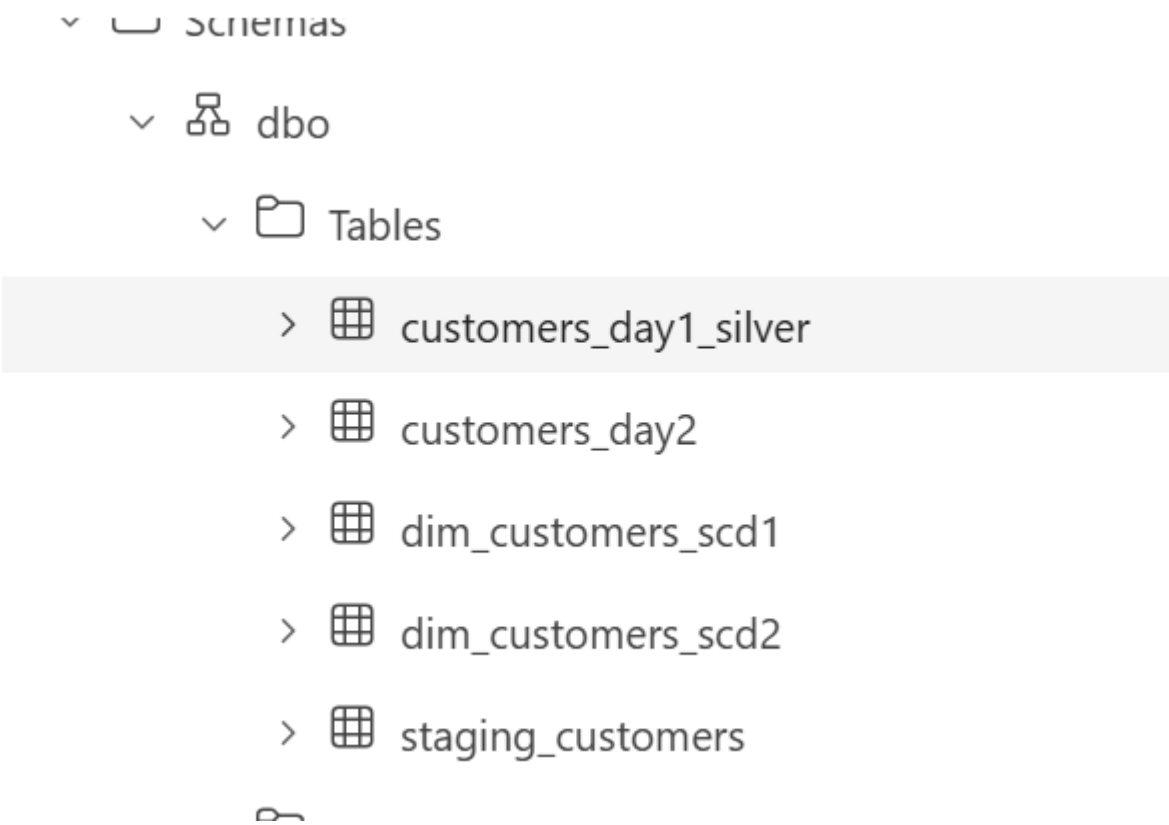
The transformed Silver data was loaded into the Warehouse using Data Pipeline (Copy Activity).

- Silver tables → Staging table (staging_customers)









Inserting details from the Silver layer into the created Stagging table:

```sql
INSERT INTO staging_customers
SELECT
    customer_id,
    name,
    city,
    email,
    updated_at
FROM customers_day1_silver;

SELECT * FROM staging_customers;
```

6. SCD TYPE 1 IMPLEMENTATION:

6.1 Dimension Table Creation

A dimension table was created to store the latest version of data without maintaining history.

6.2 Initial Load

Customer Day 1 data was inserted into the dimension table from the staging table.

```sql
CREATE TABLE dim_customers_scd1 (
    customer_id INT,
    name VARCHAR(100),
    city VARCHAR(100),
    email VARCHAR(100),
    updated_at DATETIME2(6)
);


INSERT INTO dim_customers_scd1
SELECT
    customer_id,
    name,
    city,
    email,
    updated_at
FROM staging_customers;

SELECT * FROM dim_customers_scd1;
```

6.3 MERGE Logic

SCD Type 1 was implemented using a MERGE statement:

- Existing records were updated
- New records were inserted

```sql
MERGE INTO dim_customers_scd1 AS target
USING customers_day2 AS source
ON target.customer_id = source.customer_id

WHEN MATCHED THEN
    UPDATE SET
        target.name = source.name,
        target.city = source.city,
        target.email = source.email,
        target.updated_at = source.updated_at

WHEN NOT MATCHED THEN
    INSERT (customer_id, name, city, email, updated_at)
    VALUES (source.customer_id, source.name, source.city, source.email, source.updated_at);
```

6.4 Results

- Updated records overwrite old values
- No historical data is maintained

New records are inserted into the table:

	customer_id	name	city	email	updated_at
1	31	Customer31	Quebec	customer31@email.com	2024-01-02 10:00:00.000000
2	32	Customer32	Quebec	customer32@email.com	2024-01-02 10:00:00.000000
3	35	Customer35	Quebec	customer35@email.com	2024-01-02 10:00:00.000000
4	34	Customer34	Quebec	customer34@email.com	2024-01-02 10:00:00.000000
5	33	Customer33	Quebec	customer33@email.com	2024-01-02 10:00:00.000000

Old records are updated with the new info:

18	13	Customer13	Halifax	customer13@email.com	2024-01-02 10:00:00.000000
19	14	Customer14	Edmonton	customer14_new@email.com	2024-01-02 10:00:00.000000
20	15	Customer15	Seattle	customer15@email.com	2024-01-02 10:00:00.000000
21	16	Customer16	Toronto	customer16@email.com	2024-01-02 10:00:00.000000
22	17	Customer17	Vancouver	customer17@email.com	2024-01-02 10:00:00.000000
23	18	Customer18	Calgary	customer18@email.com	2024-01-02 10:00:00.000000
24	19	Customer19	Ottawa	customer19@email.com	2024-01-02 10:00:00.000000
25	20	Customer20	Seattle	customer20@email.com	2024-01-02 10:00:00.000000
26	21	Customer21	Halifax	customer21_new@email.com	2024-01-02 10:00:00.000000

ilot completions: Off Columns: 5 Rows: 35

7. SCD TYPE 2 IMPLEMENTATION:

7.1 Dimension Table Creation

A separate dimension table was created with additional columns:

- start_date
- end_date
- is_current

```sql
-- SCD TYPE 2 IMPLEMENTATION:

CREATE TABLE dim_customers_scd2 (
    customer_id INT,
    name VARCHAR(100),
    city VARCHAR(100),
    email VARCHAR(100),
    updated_at DATETIME2(6),
    start_date DATETIME2(6),
    end_date DATETIME2(6),
    is_current BIT
);
```

7.2 Initial Load

Customer Day 1 data was loaded into the dimension table with:

- is_current = 1
- end_date = NULL

```sql
INSERT INTO dim_customers_scd2
SELECT
    customer_id,
    name,
    city,
    email,
    updated_at,
    updated_at AS start_date,
    NULL AS end_date,
    1 AS is_current
FROM staging_customers;
```

7.3 Handling Updates

Customer Day 2 dataset was loaded into the staging table and compared with existing data.

Step 1: Close Old Records

- Matching records with changes were updated
- end_date set to new timestamp
- is_current set to 0

Step 2: Insert New Records

- New version of updated records inserted
- is_current set to 1
- start_date updated

```sql
MERGE INTO dim_customers_scd2 AS target
USING customers_day2 AS source
ON target.customer_id = source.customer_id
AND target.is_current = 1

-- 1. If record exists and data changed → close old record
WHEN MATCHED AND (
    ISNULL(target.name, '') <> ISNULL(source.name, '') OR
    ISNULL(target.city, '') <> ISNULL(source.city, '') OR
    ISNULL(target.email, '') <> ISNULL(source.email, '')
)
THEN
    UPDATE SET
        target.end_date = source.updated_at,
        target.is_current = 0
```

```sql
-- 2. If new record → insert
WHEN NOT MATCHED THEN
    INSERT (
        customer_id,
        name,
        city,
        email,
        updated_at,
        start_date,
        end_date,
        is_current
    )
    VALUES (
        source.customer_id,
        source.name,
        source.city,
```

```sql
INSERT INTO dim_customers_scd2 (
    customer_id,
    name,
    city,
    email,
    updated_at,
    start_date,
    end_date,
    is_current
)
SELECT
    s.customer_id,
    s.name,
    s.city,
    s.email,
```

```sql
    s.name,
    s.city,
    s.email,
    s.updated_at,
    s.updated_at,
    NULL,
    1
  FROM customers_day2 s
  JOIN dim_customers_scd2 d
  ON s.customer_id = d.customer_id
  WHERE d.is_current = 0
  AND d.end_date = s.updated_at;



  SELECT * FROM dim_customers_scd2;
```

7.4 Results

- Historical records preserved
- Latest records marked as current

Observations:

- Old records have is_current = 0
- New records have is_current = 1
- Full history tracking achieved

Old Records that are to be updated IsCurrent is set to 0:

16	5	Customer5	Halifax	customer5@em...	2024-01-01 10:...	2024-01-01 10:...	2024-01-02 10:...	0
17	7	Customer7	Winnipeg	customer7@em...	2024-01-01 10:...	2024-01-01 10:...	2024-01-02 10:...	0
18	10	Customer10	Calgary	customer10@e...	2024-01-01 10:...	2024-01-01 10:...	2024-01-02 10:...	0
19	14	Customer14	Edmonton	customer14@e...	2024-01-01 10:...	2024-01-01 10:...	2024-01-02 10:...	0
20	15	Customer15	Winnipeg	customer15@e...	2024-01-01 10:...	2024-01-01 10:...	2024-01-02 10:...	0
21	20	Customer20	Montreal	customer20@e...	2024-01-01 10:...	2024-01-01 10:...	2024-01-02 10:...	0
22	21	Customer21	Halifax	customer21@e...	2024-01-01 10:...	2024-01-01 10:...	2024-01-02 10:...	0

The old records that match with the new source are updated and the new ones IsCurrent is set to 1:

ssages Results

customer_id	name	city	email	updated_at	start_date	end_date	is_current
5	Customer5	Seattle	customer5@em...	2024-01-02 10:...	2024-01-02 10:...	NULL	1
7	Customer7	Winnipeg	customer7_new...	2024-01-02 10:...	2024-01-02 10:...	NULL	1
10	Customer10	Seattle	customer10@e...	2024-01-02 10:...	2024-01-02 10:...	NULL	1
14	Customer14	Edmonton	customer14_ne...	2024-01-02 10:...	2024-01-02 10:...	NULL	1
15	Customer15	Seattle	customer15@e...	2024-01-02 10:...	2024-01-02 10:...	NULL	1
20	Customer20	Seattle	customer20@e...	2024-01-02 10:...	2024-01-02 10:...	NULL	1
21	Customer21	Halifax	customer21_ne...	2024-01-02 10:...	2024-01-02 10:...	NULL	1
25	Customer25	Seattle	customer25@e...	2024-01-02 10:...	2024-01-02 10:...	NULL	1

completions: Off Columns: 8 Rows

New Records that are Inserted:

31	Customer31	Quebec	customer31@e...	2024-01-02 10:...	2024-01-02 10:...	NULL	1
32	Customer32	Quebec	customer32@e...	2024-01-02 10:...	2024-01-02 10:...	NULL	1
35	Customer35	Quebec	customer35@e...	2024-01-02 10:...	2024-01-02 10:...	NULL	1
34	Customer34	Quebec	customer34@e...	2024-01-02 10:...	2024-01-02 10:...	NULL	1
33	Customer33	Quebec	customer33@e...	2024-01-02 10:...	2024-01-02 10:...	NULL	1

8. CHALLENGES FACED:

- No access to ADLS Gen2
- On-premises SQL connection issues due to gateway limitations

Solutions:

- Used Lakehouse as source system
- Simulated real-time data using multiple files

9. CONCLUSION:

This project successfully demonstrates the implementation of a modern data engineering pipeline using Microsoft Fabric. Despite infrastructure limitations, all key concepts such as data ingestion, transformation, and SCD implementations were achieved.

SCD Type 1 and Type 2 were implemented effectively using SQL MERGE and additional logic. The project highlights the importance of handling changing data in data warehousing and provides a strong foundation for real-world data engineering solutions.